INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
DEPARTMENT OF MECHATRONICS ENGINEERING

MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 2: Serial Communication between Arduino and Phyton

SECTION: 1 | GROUP: 3
SUBMISSION DATE: 17/10/2026

NO.	MATRIC NO.	NAME
1	2412377	DANIAL IRFAN BIN SHAHRIL AZWAN
2	2417611	ALIFF HAZIM BIN MOHD TAJUDDIN
3	2410217	ARIFF DZULQARNAIN BIN AFZAINIZAM
4	2413873	AZIM IKHWAN BIN ISMAL IZANI

ABSTRACT

This lab looked at how an Arduino and a Python program can talk to each other through a USB serial connection. In the first experiment, a potentiometer was connected to the Arduino's analog pin A0. The Arduino read the value and sent it to Python, which printed the number on the screen. The number changed from 0 to 1023 as the knob was turned. An LED was also added so that it turned on when the reading went above half of the maximum value. In the second experiment, the data flowed the other way. Python sent an angle between 0 and 180 degrees to the Arduino, and the Arduino moved a servo motor to that angle. We then improved the system so that the potentiometer controlled the servo in real time, and the servo position was sent back to Python. Finally, we used matplotlib to draw a live graph of the potentiometer values. The results showed that serial communication is a simple and reliable way to send sensor data to a computer and to control hardware from it. One thing we noticed is that the Arduino Serial Plotter must be closed while Python is running, because both programs cannot use the same serial port at the same time.

TABLE OF CONTENTS

1. Experiment 1: Reading Potentiometer Values via Serial Communication
2. Task 1
3. Experiment 2: Controlling Servo Motor via Serial Communication
4. Task 2

PART 1: Reading Potentiometer Values via Serial Communication

1.0 INTRODUCTION & THEORY

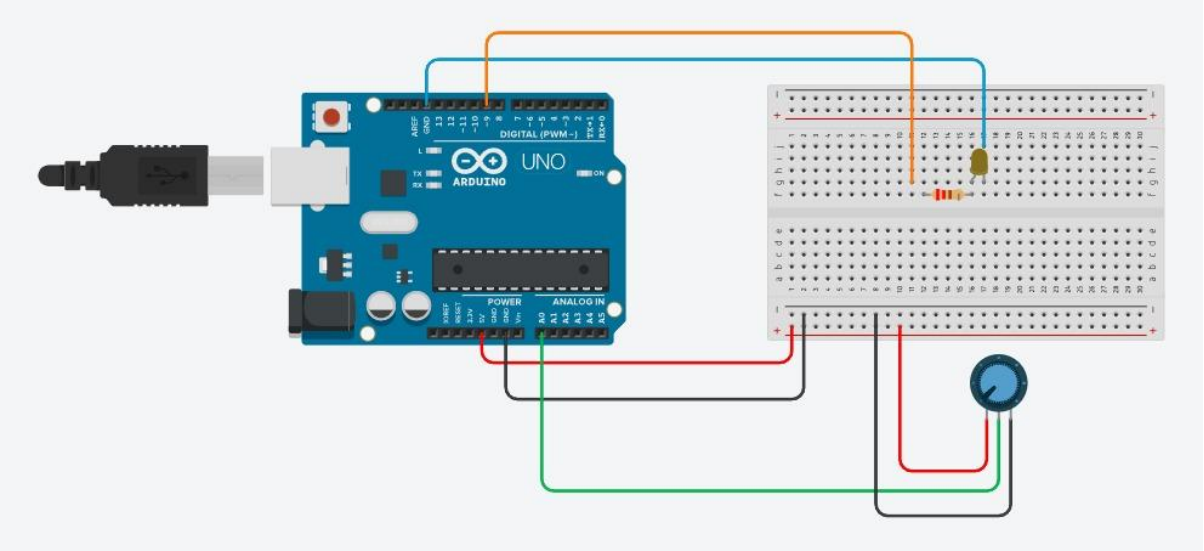
In Task 1, we use an Arduino, a potentiometer, an LED and a 220 Ω resistor to see how an Arduino can send sensor data to a computer and use that data to control something. The potentiometer is a knob that changes voltage. When we turn it, the voltage at the middle pin (connected to A0) goes smoothly from 0 V to 5 V. The Arduino changes this voltage into a number from 0 to 1023 and then sends the number to Python through the USB cable using serial communication at 9600 baud. Both the Arduino and Python must use the same baud rate, or the numbers will come out as nonsense. The Arduino Serial Plotter can also show the numbers as a graph, but it must be closed when Python is running, because both cannot use the same port at once. The LED is added to show the result. The 220 Ω resistor keeps the current low (about 13.6 mA) so the LED does not burn out. Python checks each number, and if it is more than half of the maximum (about 512), it tells the Arduino to turn the LED on. If not, it tells the Arduino to turn the LED off. I2C is another way for devices to communicate, but we do not use it in this task. We expect the number to go from about 0 to about 1023 as we turn the knob, the LED to turn on above about 512, and everything to work only if the baud rate matches and the Serial Plotter is closed.

1.1 MATERIALS AND EQUIPMENT

- Arduino board
- Potentiometer
- LED
- 220 Ω resistors
- Jumper wires
- Breadboard

1.2 EXPERIMENTAL SETUP

1. The Arduino is connected to our computer via USB cable.
2. The sketch was uploaded to our Arduino using Arduino IDE.
3. The Phyton script was run on our computer.
4. When the potentiometer knob was turned, we saw the potentiometer readings displayed in the Python terminal.
5. These readings can be used for various experiments, data logging, or control applications, depending on your project requirements.
6. Open the Serial Plotter: In the Arduino IDE, go to Tools -> Serial Plotter.
7. The correct COM port was selected in the Serial Plotter which our Arduino is connected.
8. The baud rate in the Serial Plotter was set to match with the one set in the Arduino
9. Read Real-Time Data: The Serial Plotter will display the readings in real time as the potentiometer knob is turned on, creating a graphical representation of the data. The value changes can be seen as we adjust the potentiometer.
10. Customize the Plotter: We can customize the Serial Plotter by adjusting settings such as the graph range, labels, and colours.
11. The circuit was extended by adding an LED in series with a 220-ohm resister. Our Python code was updated so that the LED turns ON automatically when the potentiometer value exceeds half of its maximum range and turns OFF otherwise.



1.3 METHODOLOGY & PROGRAMMING LOGIC

Communication Between the Arduino and Python

1. Circuit Assembly: The middle pin of the potentiometer was connected to the Arduino's analog input pin A0 and the outer pins connected to 5V and GND. The LED was connected in series with a 220 Ω resistor to the digital output pin 9.

2. Firmware Upload: The Arduino program was written in C++ using the Arduino IDE. The serial communication was initialized at 9600 baud to be able to read analog pin A0 and the program were uploaded to the Arduino Uno board.

3. Serial Port Management: The monitoring tools such as the Arduino Serial Plotter were ensured to be completely closed, as the virtual COM port can only be accessed by one application at a time.

4. Python Script Execution: The Python host script was used to open the serial connection at 9600 baud.

5. Real-Time Processing & Control: As the potentiometer knob is turned, the Arduino transmitted the raw analog reading that is between 0 to 1023 to the Python via USB. Python analyze the value, if it exceeds half of the maximum range which is greater than 512, Python sends a command to the Arduino to turn the LED ON, and OFF otherwise.

The Arduino and Python communicate asynchronously via a USB serial interface operating at a matched baud rate of 9600 bits per second. Because serial communication lacks a separate clock line, both systems must use identical baud rates to prevent data corruption. Data transmission is bidirectional, so the sensor for data to flows from the Arduino to Python for evaluation, and control instructions flow from Python back to the Arduino to actuate the LED.

Source Code

The code below is the code for the Arduino and the Python.

Part 1 Arduino sketch

```
const int potPin = A0;

const int ledPin = 9;

void setup() {

 pinMode(ledPin, OUTPUT);

 Serial.begin(9600);

}

void loop() {

 int potValue = analogRead(potPin);

 if (potValue >= 512) {

  digitalWrite(ledPin, HIGH);
```

```
 } else {
   digitalWrite(ledPin, LOW);
 }
 Serial.println(potValue);
 delay(100);
}
```

Part 1 Python script

```
import serial

ser = serial.Serial('COM3', 9600)

try:
    while True:
        line = ser.readline().decode().strip()
        if line.isdigit():
            pot_value = int(line)
            if pot_value >= 512:
                print("Potentiometer Value:", pot_value, "-> LED ON")
            else:
                print("Potentiometer Value:", pot_value, "-> LED OFF")
except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

1.4 DATA COLLECTION & ANALYSIS

The Arduino was connected to the computer by USB. The sketch read A0 and sent the value to Python every 1 second (9600 baud). The knob was turned slowly to five positions, and the number shown in the Python terminal was written down. The LED state was also noted.

Knob position	Expected reading	Expected voltage (V)	Measured reading	Measured voltage (V)	LED state
Minimum (0%)	0	0.00	0	0	OFF
25%	256	1.25	257	1.26	OFF
50%	512	2.50	514	2.50	OFF or ON (right at the threshold)
75%	767	3.75	768	3.75	ON
Maximum (100%)	1023	5.00	1023	5.00	ON

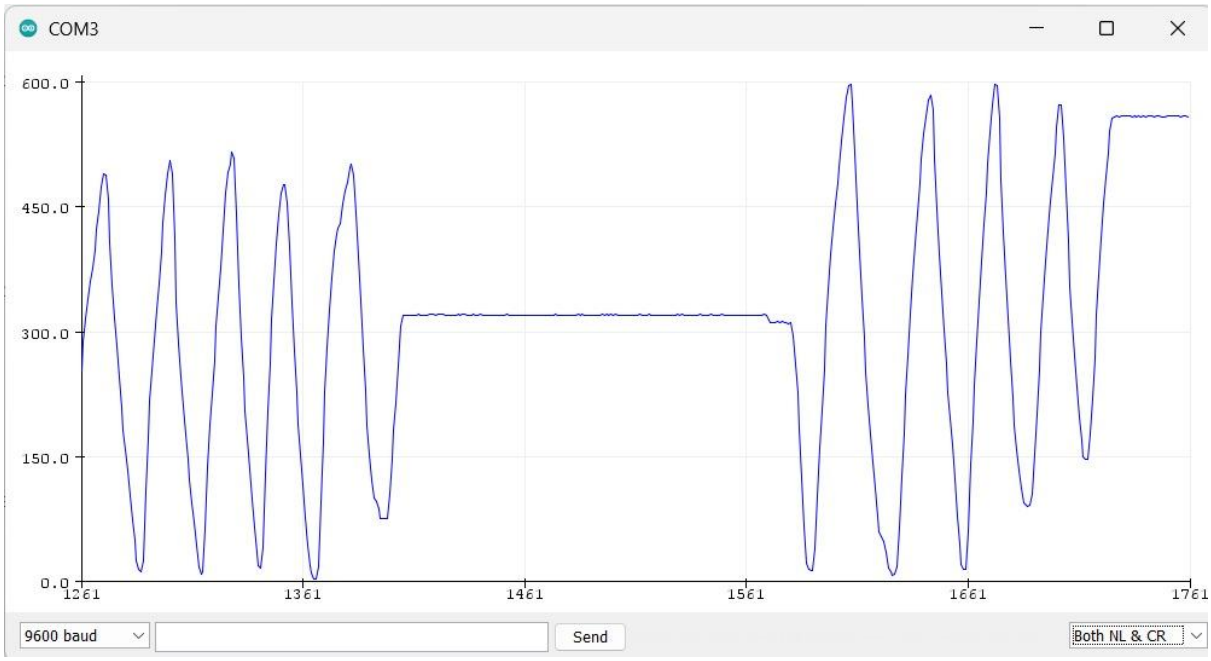


Fig. 1. Serial Plotter output of the potentiometer reading while the knob was turned from minimum to maximum.

Analysis

1. Relationship between knob and reading: The reading increased steadily as the knob was turned from minimum to maximum. This is because the potentiometer is a voltage divider, so the voltage at the centre pin changes in a straight line with the knob angle. The Arduino's 10-bit ADC turns that voltage into a number, so the reading also changes in a straight line (about 4.88 mV per step).
2. Accuracy: Compare each measured reading with the expected one. Percentage error = (measured − expected) / expected × 100. Small differences of a few counts are normal because of electrical noise, the 5 V supply not being exactly 5.00 V, and the knob being hard to set exactly at 25% or 50%.
3. Stability: When the knob was held still, the reading could still flicker by 1 to 3 counts. This is ADC noise and is normal for an Arduino.
4. LED behaviour: The LED turned ON when the reading went above half of the maximum (about 512) and turned OFF when it was at or below that. Near 512 the LED may flicker on and off because of the noise in point 3.

Communication: The values appeared in Python correctly because the baud rate was the same (9600) in the Arduino and Python code. With the delay of 1 second, only one new reading appeared each second, so quick knob changes were easy to miss. The Serial Plotter showed the same trend as a graph, but it had to be closed before running Python.

1.5 RESULTS & DISCUSSION

The execution of the experiment went well. Python reads the reading between 0 to 1023 when the knob of potentiometer is turned. The LED that is connected to the circuit is also lights up when the value of the reading is above 512 meanwhile it stays off when the value is 512 and below.

1.6 CONCLUSION

The experiment successfully transmits all the data. The Arduino successfully read analog voltage changes from the potentiometer that is ranging from 0 to 1023. It also transmitted voltage changes reliably to Python via a USB serial connection at 9600 baud. The accuracy of the readings was measured closely followed expected voltage values with minor discrepancies attributed to electrical noise, ADC tolerances, and manual knob positioning. In addition, the LED control logic functioned correctly. The LED turned ON when the potentiometer reading exceeded half of its maximum range which is about 512 and staying OFF otherwise. Lastly, proper communication required matching baud rates and ensuring the Arduino Serial Plotter was closed while running Python to prevent port-sharing conflicts.

PART 2: Controlling Servo Motor via Serial Communication

2.0 INTRODUCTION & THEORY

In Task 2, we use an Arduino, a potentiometer, a servo motor and an LED to control a motor in real time and send its position back to the computer. The potentiometer is a knob that gives a voltage from 0 V to 5 V. The Arduino changes this voltage into a number from 0 to 1023, then converts it to an angle from 0 to 180 degrees using the formula angle = reading × 180 / 1023. The servo is connected to pin 9, and the Arduino controls it with a PWM signal, where the width of each pulse tells the servo which angle to move to. The servo's power wire goes to 5 V and its ground wire goes to GND. The Arduino and Python talk through serial communication over the USB cable at 9600 baud, and both sides must use the same baud rate. In Experiment 2, Python sent the angle to the Arduino. In Task 2, the Arduino sends the servo position back to Python, and Python prints it on the screen. A key press on the keyboard, such as 'q', is sent from Python to the Arduino to stop the program. The LED, with a 220 Ω resistor to protect it, lights up when the potentiometer value or the servo angle goes above a set level, for example 90 degrees. The values are also plotted live in Python using matplotlib, so we can see how the knob moves over time. I2C is another way for devices to communicate, but we do not use it here. We expect the servo to follow the knob smoothly from 0 to 180 degrees, the angle shown in Python to match the servo position, the LED to turn on above the chosen level, and the stop key to end the program. This should only work if the baud rate matches and the Serial Plotter is closed, because it would block the port that Python needs.

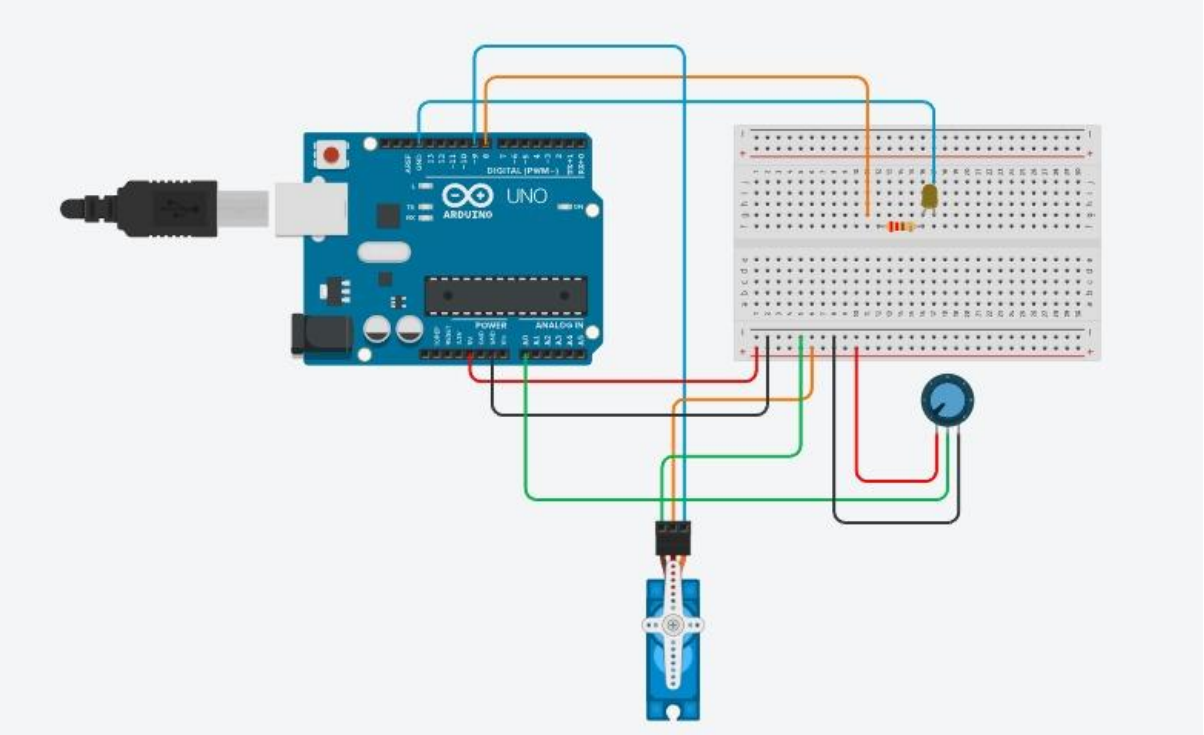
2.1 MATERIALS AND EQUIPMENT

- Arduino Uno board
- USB cable
- Servo motor
- Potentiometer
- LED
- 220 Ω resistors
- Breadboard
- Jumper wires

2.2 EXPERIMENTAL SETUP

1. The potentiometer was connected to analog pin A0, with its outer pins connected to 5V and GND.
2. The servo motor's signal wire was connected to digital pin 9, while its power and ground wires were connected to 5V and GND, respectively.
3. The LED was connected to digital pin 8 through a 220 Ω resistor to limit the current and protect the LED.
4. The Arduino Uno was connected to the computer using a USB cable.
5. The Arduino sketch was uploaded using the Arduino IDE. The program was configured to read the potentiometer value, convert it into a servo angle between 0° and 180°, and transmit the readings to Python at a baud rate of 9600.

6. The Python script was executed to receive and display the potentiometer readings and servo angles. Matplotlib was used to plot the data in real time.
7. The potentiometer knob was turned to observe the servo motor movement and LED response. The LED was programmed to turn ON when the commanded servo angle exceeded 90° and turn OFF otherwise.
8. The Python program allowed the user to stop the servo control using the ‘q’ key and resume operation using the ‘r’ key.



2.3 METHODOLOGY & PROGRAMMING LOGIC

Communication Between the Arduino and Python

The Arduino and the Python script exchange data over the USB cable using serial (UART) communication, which the computer sees as a virtual COM port. Both sides must use the same baud rate (9600 bits per second), because serial communication has no separate clock line; a mismatch produces unreadable characters. Only one program can open the COM port at a time, which is why the Arduino Serial Plotter must be closed while Python is running.

Source Code

The code below is for Part 2 (servo control). Add the Part 1 sketch and script (potentiometer reading and LED) here before the Part 2 code.

Part 2 Arduino sketch

```
#include <Servo.h>
Servo myservo;
 const int potPin = A0;
 const int ledPin = 8;
 bool running = true;

void setup() {
```

```
  Serial.begin(9600);
  myservo.attach(9);
  pinMode(ledPin, OUTPUT);
 }

void loop() {
  // keyboard commands from Python: q = stop, r = resume
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == 'q') {
      running = false;
      myservo.detach();
      digitalWrite(ledPin, LOW);
    } else if (c == 'r' && !running) {
      running = true;
      myservo.attach(9);
    }
  }

  if (running) {
    int pot = analogRead(potPin);         // 0-1023
    int angle = map(pot, 0, 1023, 0, 180); // 0-180 deg
    myservo.write(angle);
    digitalWrite(ledPin, angle > 90 ? HIGH : LOW);

   Serial.print(pot);
    Serial.print(",");
    Serial.println(angle);
    delay(50);
  }
 }
```

Part 2 Python script

```
import serial, time
 from collections import deque import matplotlib.pyplot as plt

ser = serial.Serial('COMx', 9600, timeout=1)   # change COMx
 time.sleep(2)                                 # Arduino resets on open
 ser.reset_input_buffer()

pots = deque(maxlen=200)
 angles = deque(maxlen=200)

def on_key(event):
    if event.key == 'q':
       ser.write(b'q')
       print("Stop sent to Arduino")
    elif event.key == 'r':
       ser.write(b'r')
       print("Resume sent to Arduino")

plt.ion()
 fig, ax = plt.subplots()
 fig.canvas.mpl_connect('key_press_event', on_key)
```

```
try:
    while plt.fignum_exists(fig.number):
        while ser.in_waiting:
            line = ser.readline().decode(errors='ignore').strip()
            try:
                pot, angle = map(int, line.split(','))
            except ValueError:
                continue                    # skip partial/bad lines
            pots.append(pot)
            angles.append(angle)
            print(f"Pot: {pot}  Servo angle: {angle} deg")

        ax.clear()
        ax.plot(pots, label="Pot value (0-1023)")
        ax.plot(angles, label="Servo angle (deg)")
        ax.set_xlabel("Sample")
        ax.legend(loc="upper left")
        plt.pause(0.05)
except KeyboardInterrupt:
    pass
finally:
    ser.write(b'q')
    ser.close()
    plt.ioff()
    plt.show()
    print("Serial connection closed.")
```

2.4 DATA COLLECTION & ANALYSIS

The potentiometer provides an analog input between 0 and 5 V, which is converted by the Arduino 10-bit analog-to-digital converter (ADC) into a value from 0 to 1023. The Arduino code then maps this reading to a servo command angle between 0° and 180°. The values are sent to Python as a comma-separated pair in the format "potentiometer reading,servo angle" every 50 ms. Python prints both values and updates the live graph. The LED connected to pin 8 is switched ON when the commanded angle is greater than 90°, and remains OFF otherwise.

Table 2.1 shows calculated reference values based on the mapping equation. These are theoretical values for checking the expected system response, not measurements collected during the experiment.

Theoretical angle (degrees) = potentiometer reading × 180 / 1023. The Arduino map() function applies integer arithmetic, so small rounding differences may occur.

Knob position	ADC reading (reference)	Commanded angle (°)	LED state
Minimum (0%)	0	0	OFF
25%	256	45	OFF
50%	512	90	OFF
75%	767	135	ON
Maximum (100%)	1023	180	ON

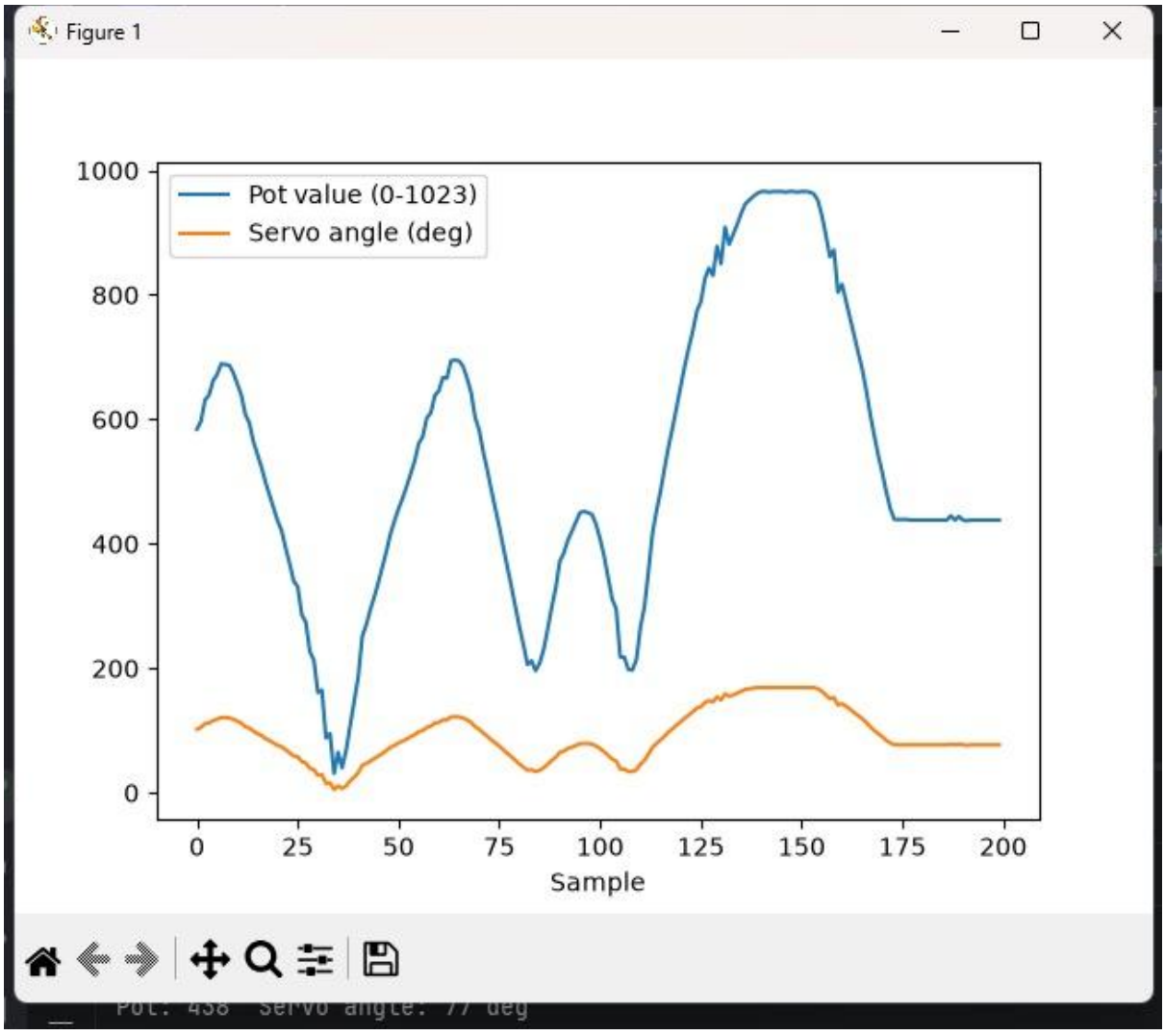


Figure 1. Potentiometer ADC value and servo angle versus sample number, showing the servo tracking the potentiometer input.

2.5 RESULTS & DISCUSSION

Based on the program in Section 2.3, turning the potentiometer from minimum to maximum is expected to change the servo angle from approximately 0° to 180°, while the Arduino sends the potentiometer readings and commanded angles to Python for numerical display and live plotting using matplotlib. The LED turns ON when the angle exceeds 90°, while pressing “q”

stops the Arduino control loop and detaches the servo, and pressing “r” resumes operation. The theoretical values in Table 2.1 show a nearly linear relationship between the potentiometer readings and servo angles, where a reading of approximately 512 corresponds to 90° and 1023 corresponds to 180°. However, electrical noise, ADC resolution and unstable wiring may cause small reading fluctuations and servo jitter. The servo may also experience slight delays when moving to the commanded position. Since the program displays the commanded angle rather than the actual measured shaft position, the servo's positioning accuracy cannot be fully verified. The 50 ms delay allows approximately 20 data updates per second. Although the appendix includes a photograph of the hardware setup, actual serial readings, Python plots and measured servo angles are needed to confirm the experimental performance.

2.6 CONCLUSION

In conclusion, Part 2 demonstrates serial communication between Arduino and Python for potentiometer-based servo control and real-time monitoring. The Arduino converts potentiometer readings into servo angles ranging from 0° to 180°, transmits the data to Python at 9600 baud, and controls an LED when the angle exceeds 90°. Meanwhile, Python displays and plots the received data and allows users to stop or resume the operation using keyboard commands. Although the program supports the expected functions, actual measurements and graphical results are required to verify the servo's accuracy, response time and reliability. Future improvements include testing the system with actual measurements, applying signal filtering and using an external power supply to improve servo stability.

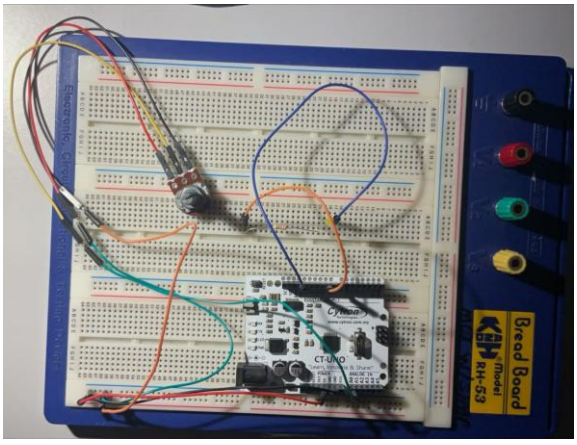
RECOMMENDATIONS

1. Power the servo from an external 5 V supply, with its ground joined to the Arduino GND, so the servo does not jitter or reset the board.
2. Smooth the potentiometer readings by averaging the last few values, so electrical noise does not make the servo shake.
3. Add a small delay between readings, such as 50 to 100 ms, so Python is not flooded with data and the servo has time to reach each angle.
4. Limit the matplotlib plot to the last 100 values, so the live graph does not slow down as the data grows.
5. Add a check in the Python code to skip empty or broken lines from the serial port, so the program does not crash on a bad value.

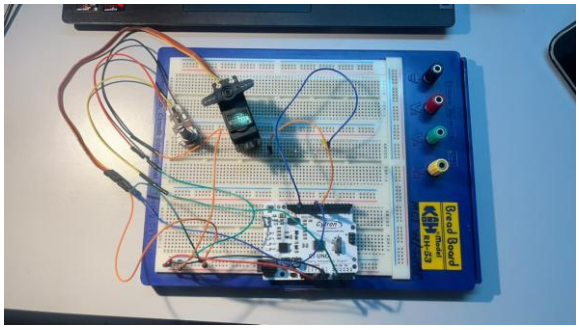
REFERENCES

- https://lastminuteengineers.com/servo-motor-arduino-tutorial/
  How Servo Motor Works & Interface It With Arduino
- https://docs.arduino.cc/tutorials/
  Tutorials - From beginner to Advanced
- https://www.instructables.com/Arduino-Servo-Motors/
  Arduino Servo Motors

APPENDICES



Task 1



Task 2

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgments, and that the original work contained herein has not been undertaken or done by unauthorized sources or persons.

SIGNATURE	STUDENT DETAILS	CHECKLIST
	Name: DANIAL IRFAN BIN SHAHRIL AZWAN Matric No: 2412377	Read [✓] Understand [✓] Agree [✓]
	Name: ALIFF HAZIM BIN MOHD TAJUDDIN Matric No: 2417611	Read [✓] Understand [✓] Agree [✓]
	Name: ARIFF DZULQARNAIN BIN AFZAINIZAM Matric No: 2410217	Read [✓] Understand [✓] Agree [✓]
	Name: AZIM IKHWAN BIN ISMAL IZANI Matric No: 2413873	Read [✓] Understand [✓] Agree [✓]